МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №5

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод Ньютона

Студентка гр. 4342 Волкова В.Н.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Используя программы-функции NEWTON и ROUND из файла methods.cpp, найти корень уравнения f(x) = 0 с заданной точностью Eps методом Ньютона, исследовать скорость сходимости и обусловленность метода.

Основные теоретические положения.

В случае, когда известно хорошее начальное приближение решения уравнения f(x) = 0, эффективным методом повышения точности является метод Ньютона. Он состоит в построении итерационной последовательности $x_{n+1} = x_n - f(x_n) / f'(x_n)$, сходящейся к корню уравнения f(x) = 0.

Для сходимости метода Ньютона на отрезке [a, b] обычно требуют, чтобы функция f(x), а также ее первая и вторая производные были непрерывны на [a, b], производная f′(x) не обращалась в ноль, а вторая производная сохраняла постоянный знак. Начальное приближение x_0 выбирают так, чтобы выполнялось условие $f(x_0)f''(x_0) > 0$.

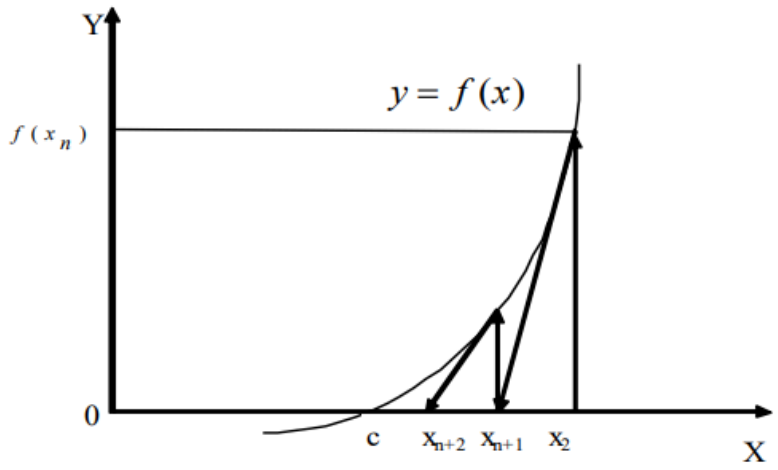

Рис.2

Метод Ньютона допускает простую геометрическую интерпретацию (рис. 2). Если через точку с координатами (x_n ; $f(x_n)$) провести касательную, то абсцисса точки пересечения этой касательной с осью Ox будет очередным приближением x_{n+1} корня уравнения f(x) = 0.

Для оценки погрешности n-го приближения корня предлагается пользоваться неравенством

$$|c - x_n| \le \frac{M_2}{2m_1}|x_n - x_{n-1}|^2,$$

где M2-наибольшее значение модуля второй производной |f″(x)| на отрезке [a,b]; m1- наименьшее значение модуля первой производной |f′(x)| на отрезке [a,b]. Таким образом, если

$$|x_n - x_{n-1}| < \varepsilon, \text{ то } |c - x_n| \le \frac{M_2\varepsilon^2}{2m_1}.$$

Это означает, что при хорошем начальном приближении корня после каждой итерации число верных десятичных знаков в очередном приближении удваивается, т.е. процесс сходится очень быстро (имеет место квадратическая сходимость). Из указанного следует, что при необходимости нахождения корня с точностью итерационный процесс можно прекращать, когда

$$|x_n - x_{n-1}| < \varepsilon_0 = \sqrt{2m_1\varepsilon / M_2}. \quad (3.1)$$

Рассмотрим один шаг итераций. Если на (n-1)-м шаге очередное приближение x_{n-1} не удовлетворяет условию окончания процесса, то вычисляются величины $f(x_{n-1})$ $f'(x_{n-1})$, и следующие приближение корня $x_n = x_{n-1} - f(x_{n-1}) / f'(x_{n-1})$. При выполнении условия (3.1) величина x_n принимается за приближенное значение корня c, вычисленное с точностью ε.

Выполнение работы.

Вариант 2: функция $f(x) = \arccos(x^2) - x$

1. Определим отрезок [a,b]: Сначала отделим корень аналитически. Функция определена при $|x| \le 1$, то есть на промежутке [-1, 1]. На отрезке [-1, 0] имеем $\arccos(x^2) \ge 0$ и $-x \ge 0$, следовательно, $f(x) = \arccos(x^2) - x > 0$. Значит, отрицательных корней данное уравнение не имеет.

Рассмотрим промежуток [0, 0.9]. На нем функция непрерывна, а ее производные имеют вид:

$$f'(x) = -2x / \sqrt{(1 - x^4)} - 1,$$

$$f''(x) = -2(1 + x^4) / (1 - x^4)^{(3/2)}.$$

Для всех $x \in [0, 0.9]$ выполняется $f'(x) < 0$, следовательно, функция на этом промежутке строго убывает. Кроме того, $f''(x) < 0$, то есть вторая производная сохраняет знак. Проверим значения функции на концах отрезка:

$$f(0) = \pi/2 > 0, \qquad f(0.9) = \arccos(0.81) - 0.9 < 0.$$

Так как функция непрерывна и меняет знак на концах отрезка [0, 0.9], на нем существует корень уравнения $f(x) = 0$. Поскольку $f'(x) < 0$ на всем отрезке, корень единственный.

Для метода Ньютона выберем начальное приближение $x_0 = 0.9$. Это значение принадлежит отрезку [0, 0.9] и удовлетворяет условию сходимости:

$$f(0.9) \cdot f''(0.9) > 0.$$

Следовательно, на выбранном отрезке и при начальном приближении $x_0 = 0.9$ метод Ньютона применим и должен сходиться к искомому корню.

В этом можно убедиться графически, построив график f(x) (Рисунок 1).

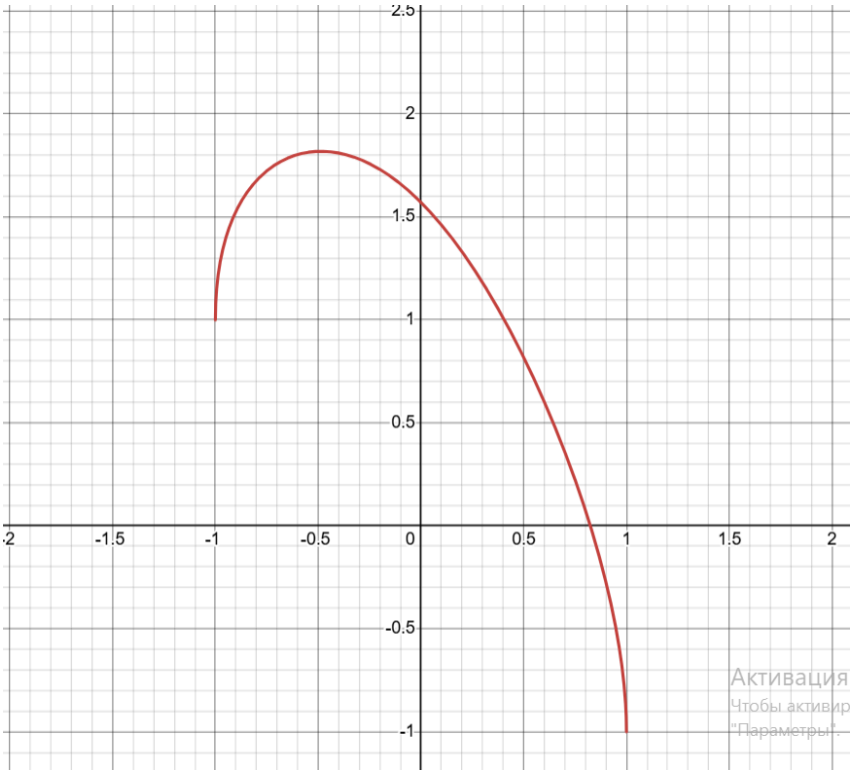

Рисунок 1 — график функции f(x)

2. Проверим условие сходимости метода Ньютона: Метод Ньютона сходится на отрезке [a,b], если функция f(x) и ее первая и вторая производные непрерывны, f′(x) не обращается в ноль, а вторая производная сохраняет постоянный знак. В нашем случае все данные условия выполняются. Следовательно, для выбранного отрезка [0,0.9] и начального приближения $x_0 = 0.9$ метод Ньютона сходится.
3. В файле lb5.cpp переопределены функции F(x) и F1(x). Первая возвращает значение $\arccos(x^2) - x$, вторая — значение производной f′(x). Если задана точность global_delta, то значения функции и производной перед возвратом округляются с помощью функции Round. Для использования метода Ньютона в начале программы определен макрос __NEWTON.
4. Создана функция int main(), где запускаются все вычисления. Они поделены на 2 части: исследование скорости сходимости метода хорд по зависимости N от Eps и исследование обусловленности метода по влиянию Delta на найденный корень.
5. Проведены вычисления, построен график скорости сходимости метода.

Рисунок 2 — график зависимости N от Eps

Из графика видно, что метод Ньютона сходится очень быстро: при уменьшении Eps от 10^{-1} до 10^{-6} число итераций увеличилось только с 1 до 4. Уже при Eps = 10^{-3} получено приближение, практически совпадающее с корнем с машинной точностью. Такое поведение соответствует квадратичной сходимости метода.

Это подтверждается и последовательностью приближений. Если принять $x^* \approx 0.8241323123$, то ошибки первых итераций составляют $|x_1 - x^*| \approx 8.69 \cdot 10^{-3}$, $|x_2 - x^*| \approx 8.83 \cdot 10^{-5}$, $|x_3 - x^*| \approx 8.89 \cdot 10^{-9}$. Ошибка уменьшается примерно как квадрат предыдущей, что характерно именно для метода Ньютона.

6. Для исследования обусловленности в функции F(x) и F1(x) было введено округление с точностью Delta. Эксперимент показывает, что при грубом округлении дальнейшее уменьшение Eps уже не улучшает найденный корень, поскольку итоговая точность начинает определяться не требуемой точностью метода, а погрешностью вычисления самой функции. Ниже в Таблице 1 приведены результаты вычислений.

Таблица 1 — исследование обусловленности метода

global_delta	ε	**значение root, возвращаемое**	**N**	**F(root)**

		HORDA		
10^{-1}	10^{-1}	0.8268292683	1	0.0000000000
	10^{-2}	0.8268292683	1	0.0000000000
	10^{-3}	0.8268292683	1	0.0000000000
	10^{-4}	0.8268292683	1	0.0000000000
	10^{-5}	0.8268292683	1	0.0000000000
	10^{-6}	0.8268292683	1	0.0000000000
10^{-3}	10^{-1}	0.8329073482	1	-0.0290000000
	10^{-2}	0.8241539525	2	0.0000000000
	10^{-3}	0.8241539525	2	0.0000000000
	10^{-4}	0.8241539525	2	0.0000000000
	10^{-5}	0.8241539525	2	0.0000000000
	10^{-6}	0.8241539525	2	0.0000000000
10^{-5}	10^{-1}	0.8328258081	1	-0.0285000000
	10^{-2}	0.8242213591	2	-0.0002900000
	10^{-3}	0.8241320289	3	0.0000000000
	10^{-4}	0.8241320289	3	0.0000000000
	10^{-5}	0.8241320289	3	0.0000000000
	10^{-6}	0.8241320289	3	0.0000000000
10^{-7}	10^{-1}	0.796824	1	-0.0285059000
	10^{-2}	0.822123	2	-0.0002868000
	10^{-3}	0.823974	3	0.0000000000
	10^{-4}	0.824103	3	0.0000000000
	10^{-5}	0.82413	3	0.0000000000
	10^{-6}	0.824132	3	0.0000000000

Из таблицы видно, что при грубом округлении Delta = 10^{-1} метод уже после первой итерации возвращает значение, для которого округленная функция формально равна нулю. Однако это не говорит о высокой точности результата: реальный корень найден лишь приблизительно. При Delta = 10^{-3} и Delta = 10^{-5} решение уточняется лучше, но дальнейшее уменьшение Eps после некоторого момента уже не влияет на итог, так как доминирует погрешность округления функции и производной.

Следовательно, метод Ньютона чувствителен к ошибкам в исходных данных. Поскольку в формуле итерации используется не только значение функции, но и ее производная, грубое округление может привести к преждевременному завершению процесса или к потере части точности результата.

7. Сравнение метода Ньютона с методами бисекции и хорд: По сравнению с методами бисекции и хорд метод Ньютона обладает наибольшей скоростью сходимости. Метод бисекции и метод хорд в данной работе имеют линейную сходимость, тогда как метод Ньютона при выполнении условий сходимости имеет квадратичную сходимость. Это позволяет получать более точное приближение к корню за меньшее число итераций. Однако метод Ньютона требует вычисления производной и более чувствителен к выбору начального приближения, тогда как метод бисекции является наиболее надежным.

Выводы.

В ходе лабораторной работы с использованием функций NEWTON и Round из файла methods.cpp был найден корень уравнения $\arccos(x^2) - x = 0$ методом Ньютона. Для данной функции выбран рабочий отрезок [0, 0.9], на котором выполняются условия сходимости метода: функция, ее первая и вторая производные непрерывны, производная не обращается в ноль, вторая

производная сохраняет знак, а начальное приближение $x_0 = 0.9$ удовлетворяет условию $f(x_0)f''(x_0) > 0$.

Исследование скорости сходимости показало, что метод Ньютона для рассматриваемой задачи обладает очень высокой эффективностью: при уменьшении Eps от 10^{-1} до 10^{-6} число итераций увеличилось только с 1 до 4, а последовательность приближений демонстрирует квадратичную сходимость.

Исследование обусловленности показало, что точность найденного корня ограничивается точностью вычисления функции и производной. При грубом округлении дальнейшее уменьшение Eps перестает улучшать ответ, поэтому для получения надежного результата необходимо выбирать Delta значительно меньше требуемой точности Eps.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lb5.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>

#define __NEWTON
#include "methods.h"

using namespace std;

double global_delta = 0.0;

double F(double x)
{
    double y = acos(x * x) - x;
    if (global_delta > 0.0) return Round(y, global_delta);
    return y;
}

double F1(double x)
{
    double y = -(2.0 * x) / sqrt(1.0 - x * x * x * x) - 1.0;
    if (global_delta > 0.0) return Round(y, global_delta);
    return y;
}

int main()
{
    double x0 = 0.9;
    int N;

    cout << fixed << setprecision(10);

    cout << "Зависимость N от Eps" << endl;
    for (double eps = 0.1; eps >= 1e-6; eps /= 10.0)
    {
        double root = NEWTON(x0, eps, N);
        cout << "Eps: " << eps
             << "\tN: " << N
             << "\tRoot: " << root
             << "\tF(root): " << F(root) << endl;
    }

    cout << endl << "Влияние Delta" << endl;
    global_delta = 1e-7;
    cout << "Delta: " << global_delta << endl;
    for (double eps = 0.1; eps >= 1e-6; eps /= 10.0)
    {
        double root = NEWTON(x0, eps, N);
        cout << "Eps: " << eps
                << "\tN: " << N
                << "\tRoot: " << root
                << "\tF(root): " << F(root) << endl;
    }
```

```
    return 0;
}
```

Файл methods.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

extern double F(double);

double BISECT(double Left,double Right,double Eps,int &N)
{
  double E = fabs(Eps)*2.0;
  double FLeft = F(Left);
  double FRight = F(Right);
  double X = (Left+Right)/2.0;
  double Y;

  if (FLeft*FRight>0.0) {puts("Неверное задание интервала\n");exit(1);}
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit(1);}

  N=0;
  if (FLeft==0.0)  return Left;
  if (FRight==0.0) return Right;

  while ((Right-Left)>=E)
  {
      X = 0.5*(Right + Left);     /* вычисление середины отрезка       */
      Y = F(X);
      if (Y == 0.0) return (X);
        if (Y*FLeft < 0.0)
           Right=X;
        else
         { Left=X; FLeft=Y; }
        N++;
  };
  return(X);
}

double Round (double X,double Delta)
{
 if (Delta<=1E-9) {puts("Неверное задание точности округления\n");exit(1);}
 if (X>0.0) return (Delta*(long((X/Delta)+0.5)));
    else    return (Delta*(long((X/Delta)-0.5)));
}

double ITER(double X0,double Eps,int &N)
{
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit (1);}
  double X1=F(X0);
  double X2=F(X1);
  N = 2;
  while( (X1 - X2)*(X1 - X2) > fabs((2*X1-X0-X2)*Eps) )
  {
      X0 = X1;
      X1 = X2;
      X2 = F(X1);
        N++;
  }
```

```
   return(X2);
}

#ifdef __NEWTON
double NEWTON (double X,double Eps,int &N)
{
  extern double F1 (double);
  double Y,Y1,DX;
  N=0;
  do
  {
    Y  = F(X);
    if (Y==0.0) return (X);

    Y1 = F1(X);
    if (Y1==0.0) {puts("Производная обратилась в ноль\n");exit(1);}

    DX=Y/Y1; X=X-DX; N++;
  }
  while (fabs(DX)>Eps);
  return (X);
}
#endif

double HORDA(double Left,double Right,double Eps,int &N)
{
  double FLeft = F(Left);
  double FRight = F(Right);
  double X,Y;

  if (FLeft*FRight>0.0) {puts("Неверное задание интервала\n");exit(1);}
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit(1);}

  N=0;
  if (FLeft==0.0)  return Left;
  if (FRight==0.0) return Right;

  do
  {
      X = Left-(Right-Left)*FLeft/(FRight-FLeft);
      Y = F(X);
      if (Y == 0.0) return (X);
        if (Y*FLeft < 0.0)
         { Right=X; FRight=Y; }
        else
         { Left=X; FLeft=Y; }
        N++;
  }
  while ( fabs(Y) >= Eps );

  return(X);

}
```

Файл methods.h

```
extern double F(double);
/*****************************************************************/
/*           Функция F (X) , задаваемая пользователем            */
/*****************************************************************/

#ifdef __NEWTON
```

```
extern double F1(double);
/*****************************************************************/
/*     Производная функции F (X) , задаваемая пользователем       */
/*****************************************************************/
#endif

double Round (double X,double Delta);
/*****************************************************************/
/*   Функция Round (X,Delta) , предназначена для округления       */
/*                    X с точностью Delta                          */
/*****************************************************************/

double BISECT(double Left,double Right,double Eps,int &N);
/*****************************************************************/
/*    Функция BISECT предназначена для решения уравнения F(X)=0   */
/*    методом деления отрезка пополам. Использованы обозначения:  */
/*      Left - левый конец промежутка                              */
/*      Right - правый конец промежутка                            */
/*      Eps - погрешность вычисления корня уравнения;              */
/*      N - число итераций                                         */
/*****************************************************************/

double ITER(double X0,double Eps,int &N);
/*****************************************************************/
/*    Функция ITER предназначена для решения уравнения F(X)=X     */
/*      методом простой итерации. Использованы обозначения:       */
/*      X0 - начальное приближение корня                           */
/*      Eps - погрешность вычисления корня уравнения;              */
/*      N - число итераций                                         */
/*****************************************************************/

double HORDA(double Left,double Right,double Eps,int &N);
/*****************************************************************/
/*    Функция HORDA предназначена для решения уравнения F(x)=0    */
/*      методом хорд. Использованы обозначения:                    */
/*      Left - левый конец промежутка                              */
/*      Right - правый конец промежутка                            */
/*      Eps - погрешность вычисления корня уравнения;              */
/*      N - число итераций                                         */
/*****************************************************************/

double NEWTON (double X,double Eps,int &N);
/*****************************************************************/
/*  Функция NEWTON предназначена для решения уравнения F(X)=0     */
/*        методом касательных. Использованы обозначения:          */
/*      X - начальное приближение корня                            */
/*      Eps - погрешность вычисления корня уравнения;              */
/*      N - число итераций                                         */
/*****************************************************************/

#include "methods.cpp"
```